

MoF 二阶段训练流水线

Stage 1: NFT 路由器训练 → Stage 2: 学生模型在线蒸馏

从 motivation 到算法实现的完整推导

Flow Factory

Abstract

本文系统阐述 MoF (Mixture-of-Flow) 的二阶段训练流水线:**Stage 1** (`trainers/mof/nft.py`) 通过 DiffusionNFT 在 reward 空间中优化 per-set、per-timestep、per-prompt 的 teacher 混合权重; **Stage 2** (`trainers/mof/distill.py`) 把 Stage 1 学到的 *router-induced teacher* (一个动态加权的 teacher 集成) 通过 on-policy 轨迹 MSE 蒸馏到一个独立的 student LoRA。我们给出 motivation、形式化定义、两阶段各自的损失/梯度推导、与代码一一对应的算法伪代码、关键的数值正确性不变量 (autocast cache 与 DDP 权重交换冲突) 以及完整的端到端流程图。全文与 `trainers/mof/{nft.py, distill.py, common.py}` 中的实现严格对齐。

Contents

1	Motivation	2
1.1	背景: 单阶段方法的局限	2
1.2	MoF 的解决思路	2
1.3	为什么需要二阶段?	2
2	Notation 与形式化	3
2.1	符号表	3
2.2	两种 Mixing Module	3
2.3	组合速度 (per-sample, per-set)	4
3	Stage 1: MoF-NFT 路由器训练	4
3.1	优化目标与可学习参数	4
3.2	Reward → Advantage Pipeline (Option B)	4
3.3	NFT Loss	5
3.4	β 在梯度中精确消去	5
3.5	对 mixing logits 的梯度	6
3.6	Off-Policy Old/New 结构 (EMA)	6
3.7	算法伪代码: Stage 1 NFT 训练循环	7
3.8	Sampling 阶段	7
3.9	数值正确性不变量 (HARD RULES)	8
4	The Bridge: Router-Induced Teacher	9
4.1	为什么蒸馏 $v^{\text{MoF*}}$ 是合理的	9
4.2	Stage 2 的形式化目标	10
5	Stage 2: 在线轨迹蒸馏	10
5.1	Stage 2 vs Stage 1 的根本区别	10
5.2	Sampling Phase (On-Policy Student Trajectory)	10
5.3	Optimization Phase (2-Pass 模式)	10
5.4	为什么必须 2-Pass?	11

5.5	算法伪代码: Stage 2 蒸馏循环	12
5.6	Train Timestep Indices: ODE vs SDE	13
5.7	LUT 索引语义陷阱	13
5.8	Checkpoint 兼容性校验	13
6	端到端流水线	14
6.1	流程图	14
6.2	统一伪代码	14
6.3	部署期推理	15
7	代码 $\Leftrightarrow$ 公式对照表	15
8	讨论	15
8.1	Stage 1 与 Stage 2 的训练成本对比	15
8.2	常见误区	16
8.3	何时需要其他变体	16
8.4	失败模式与排查	17
9	结论	17

1 Motivation

1.1 背景：单阶段方法的局限

我们已有 K 个针对不同 reward objective 微调的 LoRA teacher $\{v^{(k)}\}_{k=1}^K$ (如 GenEval / PickScore / OCR)，以及一组对应的 reward function $\{R_j\}$ 和按 source 划分的 prompt 集合 $\{\mathcal{P}_s\}_{s=1}^S$ 。两条传统路线如下：

- **OPD-Average / PCGrad**: 把所有 teacher 蒸馏到一个 student。最优 student velocity 为 teacher velocity 的（投影后）均值，由 Jensen 不等式必然在 in-domain 上退化 (mof_gradient_analysis.tex Proposition 3)。
- **OPD-Route by source**: student 在每个 source 上拟合对应的 in-domain teacher。等价于分别训练 K 个 student，OOD 泛化能力差。

两者的共同问题：student 的 velocity 在每个 (x, t) 点上被 teacher velocity 的几何关系直接锁定，无法利用 **trajectory** 级的 **reward** 信号在不同 **timestep** 偏向不同 **teacher**。

1.2 MoF 的解决思路

MoF 把搜索空间从 $\mathbb{R}^P$ (model weights, $P \sim 10^7-10^9$) 压缩到 $\mathbb{R}^{K \times T \times S}$ (mixing logits, $\sim 10^2$)，并直接在 **reward** 空间中优化 per-timestep 的 teacher 混合权重：

$$v_\lambda(x, t_i, p) = \sum_{k=1}^K \lambda_{k,i,s(p)} v^{(k)}(x, t_i, p), \quad \sum_k \lambda_{k,i,s} = 1, \lambda_{k,i,s} \geq 0 \quad (1)$$

时间互补性 (temporal complementarity) 允许 v_λ 严格超越任何单一 **teacher**：早期 timestep 用擅长 layout 的 teacher，晚期 timestep 用擅长 aesthetics 的 teacher，最终 reward 综合优于任意固定选择（详见 mof_gradient_analysis.tex Theorem 2）。

1.3 为什么需要二阶段？

只有 Stage 1 训练完之后，部署期的推理代价仍然沉重：每个 denoising step 要跑 K 次 **teacher forward**（甚至包括 router 网络），相比单 LoRA 推理是 $K \times$ 开销。这在生产环境不可接受。

二阶段训练的目的：把 **Stage 1** 学到的”动态加权 **teacher** 集成”固化进一个单独的 **student LoRA**。

二阶段流水线核心思想

Stage 1 : 在 reward 空间找出最优混合策略 ψ^*
(只训练 $\sim 10^2-10^6$ 个 mixing 参数)

↓ 把 ψ^* 看作一个”虚拟 teacher” $v^{\text{MoF}*}$

Stage 2 : 用 OPD 把 $v^{\text{MoF}*}$ 蒸馏到 student LoRA
(student 一次 forward 替代 K 次 teacher forward)

二阶段的优势：

1. 推理简化：部署时单次 student forward 替代 K 次 teacher forward + 1 次 router forward。
2. **Ensemble** 收益固化：把动态混合的 reward 增益编码进单一模型，Stage 1 的 reward improvement 通过 distillation 转移到 student。

3. 绕过 **MoF** 梯度消失: Stage 2 在 model-weight 空间训练, 不存在 normalized-mixing 瓶颈 (mof_per_set_analysis.tex 第 13 节)。
4. **OPD** 基础设施复用: Stage 2 沿用 trainers/opd 的 loader / forward signature cache 等基础设施, 仅替换 target 构造。

2 Notation 与形式化

2.1 符号表

符号	含义
K	Teacher 数量
T	推理 timesteps 数量 (num_inference_steps)
S	Prompt set 数量 (由 TeacherConfig.sources 决定)
B	Batch size
d	Latent 维度 $= C \times H \times W$
$x_0 \in \mathbb{R}^d$	Clean latent (来自 sampled trajectory 的最后一步)
x_{t_i}	在 timestep t_i 上的 noised / trajectory latent
σ_i	flow-match noise level at t_i , $\sigma_i \in [0, 1]$
$v^{(k)}(x, t, c)$	Teacher k 的 velocity field (frozen)
ψ	Mixing module 参数 (LUT 是 logits ℓ ; router 是网络参数)
$\lambda_{k,i,s}(\psi)$	Teacher k 在 timestep i 、set s 上的混合权重
$\bar{\psi}, \bar{\lambda}$	EMA shadow (off-policy old policy)
θ	Student LoRA 参数 (Stage 2 唯一可训)
$v^\theta(x, t, c)$	Student velocity field
τ	Softmax 温度
β	NFT positive/negative 插值系数
γ	OOD bonus 系数 (默认 0.2)
$A_{\max}$	advantage clip 上界 (adv_clip_range[1])

2.2 两种 Mixing Module

代码中 `create_mixing_module` 工厂函数支持两种实现, 二者输出统一为 $\mathbf{W} \in \mathbb{R}^{K \times B}$ (每列归一化), 下游使用方式完全一致。

LUT (MoFMixingModule) 离散查表 $\mathbf{\Lambda} \in \mathbb{R}^{K \times T \times S}$, 按整数索引 (k, t_{idx}, s) 查表后 softmax 得 $\lambda_{k,t_{\text{idx}},s}$ 。适合 prompt 类别稳定、推理步数固定的场景。

Continuous Router (MoFAdaLNRouter / MoFMLPRouter) 连续函数 $\lambda_k(t, \mathbf{c}) = \text{softmax}(f_\psi(t, \mathbf{c})/\tau)_k$, 其中 $\mathbf{c}$ 来自 prompt embeddings 的 attention pooling 或直接使用 `pooled_prompt_embeds`。adaLN 变体让 timestep 通过 (γ, β) 调制 text embedding, 符合 DiT conditioning 范式 (PixArt- α / SD3 / Flux)。

本文以下统一记 $\lambda_k(t, \mathbf{c}; \psi)$, LUT 是其特例 ($\mathbf{c}$ 退化为 source s 的 one-hot)。

2.3 组合速度 (per-sample, per-set)

设 batch 中样本 b 来自 source s_b , 混合速度为

$$v_{\lambda}^{(b)}(x_b, t_i; \psi) = \sum_{k=1}^K \lambda_{k,i,s_b}(\psi) \text{sg}\left(v^{(k)}(x_b, t_i, c_b)\right) \quad (2)$$

其中 $\text{sg}(\cdot)$ 表示 `.detach()`——**teacher velocity** 永远 **detached**, 梯度只流过 **mixing weights**。这是 Stage 1 的核心约定, 对应 `common.py:_compute_teacher_velocities` 的 `torch.no_grad()` + `output.noise_pred.detach()` 实现。

3 Stage 1: MoF-NFT 路由器训练

3.1 优化目标与可学习参数

Stage 1 的目标:

$$\psi^* = \arg \max_{\psi} \mathbb{E}_{p \sim \mathcal{D}} [R_{\text{composite}}(\text{ODE}(v_{\lambda}(\cdot; \psi); x_T, p))] \quad (3)$$

唯一可训参数: mixing module ψ (LUT 的 logits 或 router 的网络参数)。Teacher 参数全部 frozen, student LoRA 参数虽然 `requires_grad=True` (被 `use_named_parameters` 复用做 weight swap), 但因为所有 teacher velocities 都 `.detach()`, 梯度不会流过 teacher——见 `common.py:_init_optimizer` 注释。

3.2 Reward $\rightarrow$ Advantage Pipeline (Option B)

`common.py:prepare_feedback` 实现了三步 advantage 流水线, 对应 `mof_per_set_analysis.tex` Algorithm 2 (Option B):

Step 1: Per-reward, per-group 标准化。 对每个 reward $k \in \{1, \dots, |\mathcal{R}|\}$ 和每个 group g (共享同一 prompt 的 K 个 sample), 独立计算:

$$a_k(x_i) = \frac{R_k(x_i) - \mu_{k,g}}{\max(\sigma_{k,g}, \epsilon)}, \quad \text{NaN values skipped (applicable_sources filtering)} \quad (4)$$

Step 2: Advantage 空间聚合 (per-set)。 对样本 x_i (来自 source s_i , in-domain reward 记为 $\text{in}(s_i)$):

$$\mathcal{A}_{\text{combined}}(x_i) = a_{\text{in}(s_i)}(x_i) + \gamma \cdot \frac{1}{|\mathcal{R}_{\text{ood}}^{(i)}|} \sum_{j \in \mathcal{R}_{\text{ood}}^{(i)}} a_j(x_i) \quad (5)$$

Step 3: Global-std 归一化 + clipping。

$$\sigma_{\text{global}} = \text{std}(\{\mathcal{A}_{\text{combined}}(x_i)\}_{i=1}^N), \quad \mathcal{A}_{\text{final}}(x_i) = \text{clip}\left(\frac{\mathcal{A}_{\text{combined}}(x_i)}{\max(\sigma_{\text{global}}, \epsilon)}, \text{adv_clip_min}, \text{adv_clip_max}\right) \quad (6)$$

Remark 1 (为什么在 advantage 空间聚合). $\text{GenEval} \in [0, 1]$ 与 $\text{PickScore} \in [0.83, 0.95]$ 的 raw scale 差异极大。Step 1 已经把每个 reward 归一化到 $\sim N(0, 1)$, 因此 Step 2 中的 γ 具有尺度无关的语义: OOD advantage 贡献固定 γ 比例的信号强度。在 raw reward 空间聚合则需要为每对 (R_i, R_j) 单独调系数。

3.3 NFT Loss

记 advantage 经 $A_{\max} = \text{adv_clip_range}[1]$ 归一化后的"normalized advantage":

$$r = \text{clamp}\left(\frac{\mathcal{A}_{\text{final}}}{2A_{\max}} + \frac{1}{2}, 0, 1\right) \in [0, 1] \quad (7)$$

其中 $r = 1$ 对应最佳 sample, $r = 0$ 对应最差 sample。

Positive / Negative velocity (关于 v_{old} 对称)。

$$v_{i,b}^{\text{new}} = \sum_k \lambda_{k,i,s_b}(\psi) \text{sg}(v_{i,b}^{(k)}) \quad (\text{current } \psi) \quad (8)$$

$$v_{i,b}^{\text{old}} = \sum_k \lambda_{k,i,s_b}(\bar{\psi}) \text{sg}(v_{i,b}^{(k)}) \quad (\text{EMA } \bar{\psi}, \text{ detached}) \quad (9)$$

$$v_{i,b}^+ = \beta v_{i,b}^{\text{new}} + (1 - \beta) v_{i,b}^{\text{old}} \quad (10)$$

$$v_{i,b}^- = (1 + \beta) v_{i,b}^{\text{old}} - \beta v_{i,b}^{\text{new}} \quad (11)$$

Self-normalized MSE on x_0 reconstruction. 对每个 sample b 和 timestep i , flow-matching 噪声化:

$$x_{t_i,b} = (1 - \sigma_i) x_{0,b} + \sigma_i \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (12)$$

(重要: $x_{0,b}$ 是 sampled trajectory 的最后一步 clean latent; 噪声 ε 是每次 *optimize* 重新采样的, 不来自 sampling 阶段。这与 Stage 2 的 on-policy trajectory 用法不同, 见 §5。)

预测的 x_0 :

$$\hat{x}_{0,b}^\pm(v) = x_{t_i,b} - \sigma_i v \quad (13)$$

Self-normalized MSE (per-sample 标量):

$$\boxed{\mathcal{M}_{i,b}(v) = \frac{\|\hat{x}_{0,b}(v) - x_{0,b}\|_2^2}{\text{sg}\left(\frac{\|\hat{x}_{0,b}(v) - x_{0,b}\|_1}{d}\right)}} \quad (14)$$

分母是 detached 的 mean absolute error (clip 到 10^{-5}), 作为自适应归一化权重——重建误差大时分母大, 梯度自动缩小。

NFT 总 loss (per-sample, per-timestep)。

$$\boxed{\mathcal{L}_{i,b}^{\text{NFT}}(\psi) = \frac{r_b \mathcal{M}_{i,b}(v_{i,b}^+) + (1 - r_b) \mathcal{M}_{i,b}(v_{i,b}^-)}{\beta} \cdot A_{\max}} \quad (15)$$

最终 batch loss (在 T 个 timestep 上累积; 代码中通过 `accelerator.accumulate` 实现 micro-batching):

$$\mathcal{L}^{\text{NFT}}(\psi) = \frac{1}{B \cdot T} \sum_{b=1}^B \sum_{i=1}^T \mathcal{L}_{i,b}^{\text{NFT}}(\psi) \quad (16)$$

3.4 β 在梯度中精确消去

Theorem 1 (β Cancellation). $\partial \mathcal{L}_{i,b}^{\text{NFT}} / \partial v_{i,b}^{\text{new}}$ 与 β 完全无关; 只通过 self-normalization weight $w^\pm = \text{sg}(\|\hat{x}_0^\pm - x_0\|_1/d)$ 作二阶 magnitude 调制 (不影响方向)。

证明（链式法则系数）：

$$\frac{\partial v^+}{\partial v^{\text{new}}} = \beta, \quad \frac{\partial v^-}{\partial v^{\text{new}}} = -\beta \quad (17)$$

代入：

$$\frac{\partial \mathcal{L}^{\text{NFT}}}{\partial v^{\text{new}}} = \frac{A_{\max}}{\beta} \left[r \cdot \frac{\partial \mathcal{M}(v^+)}{\partial v^+} \cdot \beta + (1-r) \cdot \frac{\partial \mathcal{M}(v^-)}{\partial v^-} \cdot (-\beta) \right] = A_{\max} \left[r \frac{\partial \mathcal{M}^+}{\partial v^+} - (1-r) \frac{\partial \mathcal{M}^-}{\partial v^-} \right] \quad (18)$$

分子 β 与分母 $1/\beta$ 精确对消。详见 `nft_beta_analysis.tex`。

实践含义：调整 `nft_beta` $\in \{0.01, 0.1, 1.0\}$ 不会改变 `grad_norm` 的数量级，只改变 loss 的绝对值（scaling）。

3.5 对 mixing logits 的梯度

对 LUT 模式 $\psi = \ell \in \mathbb{R}^{K \times T \times S}$ （router 模式同理，增加一个 $\partial \ell / \partial \psi_{\text{router}}$ 因子）：

$$\boxed{\frac{\partial \mathcal{L}_{i,b}^{\text{NFT}}}{\partial \ell_{j,i,s_b}} = \frac{\lambda_{j,i,s_b}}{\tau} \mathbf{g}_{i,b}^\top (v_{i,b}^{(j)} - v_{i,b}^{\text{new}})} \quad (19)$$

其中 $\mathbf{g}_{i,b} = \partial \mathcal{L}_{i,b}^{\text{NFT}} / \partial v_{i,b}^{\text{new}} \in \mathbb{R}^d$ 。

跨 timestep / 跨 set 梯度为零（**Triple Decoupling**, `mof_per_set_analysis.tex` Corollary 1）：

$$\frac{\partial \mathcal{L}_{i,b}^{\text{NFT}}}{\partial \ell_{j,i',s'}} = 0 \quad \text{当 } i' \neq i \text{ 或 } s' \neq s_b \quad (20)$$

Remark 2 (Mean-subtraction 结构). 公式 (19) 中 $(v^{(j)} - v^{\text{new}})$ 项是 softmax 归一化约束 $\sum_k \lambda_k = 1$ 的必然结果 (Theorem `thm:fundamental_limit`)。当 teachers 共享同一 base model (LoRA 微调), $v^{(j)} - v^{\text{new}} \approx \delta^{(j)} - \bar{\delta}$, 量级仅为 base velocity 的 1-5%——这是 NFT 在 LoRA-shared-base 设定下梯度量级偏小的根因。详细分析与缓解方案见 `mof_per_set_analysis.tex` 第 13-14 节。

3.6 Off-Policy Old/New 结构 (EMA)

`sampling_context()` 在 sampling 阶段使用 EMA 权重 $\bar{\psi}$ 生成 trajectory; `optimize()` 阶段：

- **Phase 1 (precompute, no_grad)**：在 EMA 上下文中计算 v^{old} , detach 后缓存（每 timestep 一份）。
- **Phase 2 (gradient)**：用当前 ψ 重新计算 v^{new} , 与缓存的 v^{old} 一起喂入 NFT loss。

这种“sample with $\bar{\psi}$, optimize with ψ ”的拆分使 NFT 成为 off-policy——一份 sample 可被多个 inner epoch 复用，提高数据利用率。

3.7 算法伪代码：Stage 1 NFT 训练循环

Algorithm 1 MoF-NFT Optimization (`trainers/mof/nft.py:optimize`)

Require: Sampled trajectories $\{(\mathbf{x}_b, c_b, s_b, \mathcal{A}_b)\}_{b=1}^N$ from `SAMPLE`

Require: Mixing module ψ , EMA shadow $\bar{\psi}$, frozen teachers $\{v^{(k)}\}$

Require: NFT β , temperature τ , $A_{\max}$, train timesteps T

```

1: for inner_epoch = 1, ...,  $N_{\text{inner}}$  do
2:   shuffle samples
3:   for each micro-batch  $\mathcal{B}$  of size  $B$  do
4:      $x_0 \leftarrow \mathcal{B}.\text{all\_latents}[:, -1]$  ▷ Final clean latent
5:      $\{s_b\}_{b=1}^B \leftarrow \text{GETSETIDS}(\mathcal{B})$ 
6:     // Phase 1: Precompute  $v^{\text{old}}$  with EMA  $\bar{\psi}$  (no_grad)
7:     adapter.rollout()
8:      $\{t_i\}_{i=1}^T \leftarrow \text{SAMPLETIMESTEPS}(B)$  ▷ logit-normal / uniform / discrete
9:     for  $i = 1, \dots, T$  do
10:       $\sigma_i \leftarrow \text{FLOWMATCHSIGMA}(t_i)$ 
11:       $\varepsilon_i \sim \mathcal{N}(0, I)$ 
12:       $x_{t_i} \leftarrow (1 - \sigma_i)x_0 + \sigma_i\varepsilon_i$  ▷ Eq. (12)
13:       $\{v_i^{(k)}\}_{k=1}^K \leftarrow \text{TEACHERFORWARD}(x_{t_i}, t_i, c)$  ▷ See Alg. 3
14:      with sampling_context() (EMA  $\bar{\psi}$ ):  $v_i^{\text{old}} \leftarrow \sum_k \lambda_k(t_i, c; \bar{\psi}) \cdot v_i^{(k)}$ 
15:      cache  $(x_{t_i}, \sigma_i, v_i^{\text{old}}, \{v_i^{(k)}\})$ 
16:    end for
17:    // Phase 2: Compute NFT loss + backward (with grad)
18:    adapter.train()
19:    for  $i = 1, \dots, T$  do
20:      with accelerator.accumulate( $\psi$ ):
21:         $v_i^{\text{new}} \leftarrow \sum_k \lambda_k(t_i, c; \psi) \cdot \text{sg}(v_i^{(k)})$  ▷ Gradient flows here only
22:         $\mathcal{A} \leftarrow \text{clip}(\mathcal{B}.\text{advantage}, -A_{\max}, A_{\max})$ 
23:         $r \leftarrow \text{clamp}(\mathcal{A}/(2A_{\max}) + 0.5, 0, 1)$ 
24:         $v_i^+ \leftarrow \beta v_i^{\text{new}} + (1 - \beta)v_i^{\text{old}}$ 
25:         $v_i^- \leftarrow (1 + \beta)v_i^{\text{old}} - \beta v_i^{\text{new}}$ 
26:         $\mathcal{L}_i \leftarrow (r \mathcal{M}_i(v_i^+) + (1 - r) \mathcal{M}_i(v_i^-)) / \beta \cdot A_{\max}$  ▷ Eq. (15)
27:        (optional)  $\mathcal{L}_i += \beta_{\text{KL}} \cdot \|v_i^{\text{new}} - v_i^{\text{ref}}\|^2$ 
28:        accelerator.backward( $\mathcal{L}_i$ )
29:      if accelerator.sync_gradients then
30:        clip_grad_norm( $\psi$ , max_grad_norm)
31:        optimizer.step(); optimizer.zero_grad()
32:      end if
33:    end for
34:     $\bar{\psi} \leftarrow \text{EMASTEP}(\psi, \bar{\psi}, \text{decay})$  ▷ End-of-epoch EMA update

```

3.8 Sampling 阶段

NFT sampling 与 GRPO 不同，不需要 `log-probability: nft.py:build_sample_kwargs` 设置 `compute_log_prob=False` 和 `trajectory_indices=[-1]`（只保留最后一步 clean latent，节省内存）。

Algorithm 2 MoF Sampling Phase (`common.py:sample`)

Require: Per-source dataloaders, EMA $\bar{\psi}$

```
1: adapter.rollout()
2: for batch in INTERLEAVEDSOURCEITER do ▷ round-robin per-source
3:    $s \leftarrow \text{GETBATCHSETID}(\text{batch})$ 
4:   with _mof_inference_context( $s$ ): ▷ Patches adapter.forward to  $\sum_k \lambda_k v^{(k)}$ 
5:     sample_kwargs  $\leftarrow \text{BUILDSAMPLEKWARGS}(\text{batch})$  ▷ NFT: only final latent
6:     samples  $\leftarrow \text{adapter.inference}(**\text{sample\_kwargs})$ 
7:     stitch metadata (__source__, include, ...)  $\rightarrow \text{sample.extra\_kwargs}$ 
8:   end for
9:   Compute rewards on all samples; run advantage pipeline (Eqs. (4)–(6))
10: return samples (each carries advantage field)
```

3.9 数值正确性不变量 (HARD RULES)

下面两条不变量是 CLAUDE.md 列出的硬性约束，违反会导致 所有 **teacher** 静默产出相同输出。Stage 1 和 Stage 2 都必须遵守。

不变量 1: Autocast cache 与 weight swap 互斥。 `adapter.use_named_parameters(name)` 通过 `.data.copy_()` 交换权重，保留同一 `data_ptr`。autocast cache 按 `data_ptr` 索引 cast 后的 tensor，因此 swap 到 `teacher_1` 后仍服务 `teacher_0` 的 cached cast——所有 teacher 静默产出相同输出。

对策：在 weight-swap 循环外侧 disable autocast cache：

```
prev = torch.is_autocast_cache_enabled()
torch.set_autocast_cache_enabled(False)
try:
    for name in teacher_names:
        with adapter.use_named_parameters(name):
            output = adapter.forward(...)
finally:
    torch.set_autocast_cache_enabled(prev)
```

不变量 2: DDP 内部 buffer 与 weight swap 互斥 (inference path)。 DDP 在初始化时构建 gradient bucketing buffers，`.data.copy_()` 不更新这些 buffer。在 `torch.no_grad()` inference 模式下调用 DDP-wrapped module，仍然读取 stale buffer。

对策：`mof/utils.py:bypass_ddp_for_weight_swap` 在 inference 期间临时把 component 指向 unwrapped module：

```
@contextmanager
def bypass_ddp_for_weight_swap(adapter):
    unwrapped = adapter.get_component_unwrapped('transformer')
    wrapped = adapter.get_component('transformer')
    if unwrapped is not wrapped:
        adapter.set_component('transformer', unwrapped)
    try:
        yield
    finally:
        if unwrapped is not wrapped:
            adapter.set_component('transformer', wrapped)
```

两条不变量在以下三处实施：

1. `common.py:_compute_teacher_velocities` (用于 NFT optimize)
2. `common.py:_mof_inference_context` (用于 sampling)
3. `distill.py:_compute_teacher_velocities` (用于 distill 的 pre-pass)

Algorithm 3 Safe Teacher Forward (autocast + DDP invariants)

```

1: function TEACHERFORWARD( $x_t, t, c$ )
2:   prev_cache  $\leftarrow$  torch.is_autocast_cache_enabled()
3:   torch.set_autocast_cache_enabled(False)
4:   with BYPASSDDPFORWEIGHTSWAP(adapter), torch.no_grad():
5:     velocities  $\leftarrow$  []
6:     for name  $\in$  teacher_names do
7:       with adapter.use_named_parameters(name):
8:         out  $\leftarrow$  adapter.forward( $x_t, t, c$ )
9:         velocities.append(out.noise_pred.detach())
10:   torch.set_autocast_cache_enabled(prev_cache)
11:   return stack(velocities, dim=0)  $\triangleright (K, B, C, H, W)$ 
12: end function

```

4 The Bridge: Router-Induced Teacher

Stage 1 收敛后，最优 mixing 参数 ψ^* 定义了一个动态加权的虚拟 **teacher**：

Definition 1 (Router-Induced Teacher).

$$v^{\text{MoF}^*}(x, t, c) \triangleq \sum_{k=1}^K \lambda_k(t, c; \psi^*) \cdot v^{(k)}(x, t, c) \quad (21)$$

Proposition 1 (v^{MoF^*} 是合法的 flow-matching velocity field). 对任意 ψ^* ，由于 $\sum_k \lambda_k = 1$, $\lambda_k \geq 0$, v^{MoF^*} 是 K 个合法 velocity field 的逐点凸组合，仍满足连续性方程 $\partial_t p_t + \nabla \cdot (p_t v^{\text{MoF}^*}) = 0$ 的离散化形式。(mof_gradient_analysis.tex Proposition 1.)

4.1 为什么蒸馏 v^{MoF^*} 是合理的

1. **Reward** 最优性已保证： v^{MoF^*} 在 ψ^* 收敛点上达到 Stage 1 reward objective (3) 的局部最优。蒸馏目标是 保持这个 **reward** 增益，同时压缩推理代价。
2. 蒸馏从 **RL** 退化为监督学习： Stage 2 不再需要 reward evaluation——target v^{MoF^*} 是 well-defined 的 deterministic function。loss 是 vanilla MSE。
3. 绕过 **normalized-mixing** 梯度瓶颈： Stage 1 中 softmax mean-subtraction 把梯度限制在 $\delta^{(j)} - \bar{\delta}$ 量级 (mof_per_set_analysis.tex Theorem thm:fundamental_limit); Stage 2 的 student 是 unconstrained model weight，梯度量级 $\sim O(1)$ 。
4. 推理代价摊销： Stage 1 训练时 K 倍 teacher forward 是一次性投资； Stage 2 之后每一次部署推理只需 student 一次 forward。

4.2 Stage 2 的形式化目标

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{p, x_T \sim \mathcal{N}(0, I)} \left[\sum_{i=1}^T \left\| v^{\theta}(x_{t_i}^{\theta}, t_i, c) - v^{\text{MoF}^*}(x_{t_i}^{\theta}, t_i, c) \right\|_2^2 \right] \quad (22)$$

其中 $\{x_{t_i}^{\theta}\}_{i=1}^T$ 是 **student** 自己生成的轨迹：

$$x_{t_{i+1}}^{\theta} = \text{scheduler.step}(x_{t_i}^{\theta}, v^{\theta}(x_{t_i}^{\theta}, t_i, c), t_i) \quad (23)$$

这正是 OPD (On-Policy Distillation) 的范式：在 student 当前 policy 访问的 state 分布上做 distillation——保证训练分布与部署分布一致。

5 Stage 2: 在线轨迹蒸馏

5.1 Stage 2 vs Stage 1 的根本区别

	Stage 1 (NFT)	Stage 2 (Distill)
可训参数	Mixing ψ ($\sim 10^2$ – 10^6)	Student LoRA θ ($\sim 10^7$)
Loss 类型	Reward-weighted self-norm MSE on x_0	Pure MSE on velocity
需要 reward?	✓ (驱动 advantage)	✗ (target 由 ψ^* 给出)
On/Off-policy	Off-policy (EMA $\bar{\psi}$ for sampling)	On-policy (student 生成 trajectory)
Trajectory 用法	仅取最后 x_0 用于 NFT noise resample	全部 $\{x_{t_i}\}$ 都进入 loss
Teacher forward 频率	每 inner epoch 每 timestep 一次	每 batch 每 timestep 一次 (pre-pass)
梯度量级瓶颈	$\ v^{(j)} - \bar{v}\ \sim \text{LoRA residual}$	无 (model weight space)

5.2 Sampling Phase (On-Policy Student Trajectory)

`distill.py:sample` 中, student 用自己当前的权重 θ 生成完整 denoising trajectory:

$$x_T \sim \mathcal{N}(0, I) \quad (24)$$

$$\text{for } i = 0, 1, \dots, T-1: \quad (25)$$

$$v_i^{\theta} = v^{\theta}(x_{t_i}, t_i, c) \quad (26)$$

$$x_{t_{i+1}} = \text{scheduler.step}(x_{t_i}, v_i^{\theta}, t_i) \quad (27)$$

所有 $\{x_{t_i}\}_{i=0}^T$ 通过 `trajectory_indices` 全部存入 `batch["all_latents"]` (`compute_trajectory_indices` 基于 `train_timestep_indices` 选择存储位置)。

关键差异: Stage 2 sampling 不调用 `teachers`, 也不调用 `router`——纯粹 student 自演。Teacher / router 只在 `optimize` 阶段的 pre-pass 中介入。

5.3 Optimization Phase (2-Pass 模式)

`distill.py:optimize` 采用与 `trainers/opd` 完全相同的 2-pass 设计:

Pre-pass (`no_grad`, 逐 `timestep` 缓存 `target`)。

$$v_{i,b}^{\text{target}} \leftarrow \sum_{k=1}^K \lambda_k(t_i, c_b; \psi^*) \cdot \text{sg}(v^{(k)}(x_{t_i,b}, t_i, c_b)) \quad (28)$$

其中 $x_{t_i,b}$ 来自 student trajectory (`batch["all_latents"]`), ψ^* 从 `mof_state.pt` checkpoint 加载 (**frozen**)。

Pre-pass 中:

1. 对每个 training timestep i , 运行 K 次 **teacher forward** (受 Alg. 3 保护);
2. 调用 `_get_mof_weights` 获得 λ_k (router 模式: `router(t, prompt_embeds, pooled)`; LUT 模式: 按 `timestep_index` 和 `__source__` 索引);
3. 计算 v_i^{target} 并 **detach** 缓存。

Main pass (with grad, 纯 student forward + MSE)。

$$\mathcal{L}_{i,b}^{\text{distill}}(\theta) = \|v^\theta(x_{t_i,b}, t_i, c_b) - \text{sg}(v_{i,b}^{\text{target}})\|_2^2 \quad (29)$$

总 loss:

$$\mathcal{L}^{\text{distill}}(\theta) = \frac{1}{B \cdot T \cdot d} \sum_{b=1}^B \sum_{i=1}^T \mathcal{L}_{i,b}^{\text{distill}}(\theta) \quad (30)$$

代码中是 `((v_student.float() - v_target.float()) ** 2).mean()`, 即在所有维度 (包括 $d = C \cdot H \cdot W$) 上取平均。

5.4 为什么必须 2-Pass?

将 pre-pass 与 main-pass 分离是正确性约束, 不是性能优化。原因:

1. Teacher forward 必须 `use_named_parameters` swap weights;
2. Weight swap 必须搭配 `torch.no_grad() + bypass_ddp_for_weight_swap + set_autocast_cache_enabled` (不变量 §3.9);
3. 而 `accelerator.accumulate(student_module)` 上下文要求:
 - gradient flow 必须经过 wrapped DDP module (不能 bypass DDP);
 - autocast cache 必须开启 (性能要求)。

把 teacher forward (要求 bypass DDP / no autocast cache) 放进 `accumulate` 上下文 (要求 wrapped DDP / autocast cache) 会同时触发两条不变量违例——所有 teacher 静默产出相同输出, 蒸馏目标退化为单一 teacher (甚至是 student 自己)。

2-pass 设计是唯一正确路径: 把所有 teacher swap 集中到 pre-pass 内, 缓存 detached target; main-pass 只触碰 student 本身, gradient flow 完全受 DDP 控制。

5.5 算法伪代码：Stage 2 蒸馏循环

Algorithm 4 MoF On-Policy Distillation (trainers/mof/distill.py:optimize)

Require: Student trajectories $\{(\mathbf{x}^\theta, c, s)\}_{b=1}^N$ from SAMPLE (Alg. 5)

Require: Frozen mixing ψ^* (loaded from mof_state.pt)

Require: Frozen teachers $\{v^{(k)}\}_{k=1}^K$, train timestep indices $\mathcal{T}$

```

1: adapter.train()
2: for inner_epoch = 1, ..., Ninner do
3:   shuffle samples
4:   for each micro-batch  $\mathcal{B}$  of size  $B$  do
5:     idx_map  $\leftarrow \mathcal{B}.\text{latent\_index\_map}$ 
6:     // === Pre-pass: cache  $v^{\text{target}}$  (no_grad) ===
7:      $V^{\text{target}} \leftarrow []$ 
8:     for  $\hat{i} \in \mathcal{T}$  do ▷ e.g.,  $\{0, \dots, T-1\}$  for ODE
9:        $t_{\hat{i}} \leftarrow \mathcal{B}.\text{timesteps}[:, \hat{i}]$ 
10:       $x_{t_{\hat{i}}} \leftarrow \mathcal{B}.\text{all\_latents}[:, \text{idx\_map}[\hat{i]]$  ▷ Student-visited point
11:      forward_kwargs  $\leftarrow \text{BUILD\_FORWARD\_KWARGS}(\mathcal{B}, t_{\hat{i}}, x_{t_{\hat{i}}})$ 
12:       $\{v_{\hat{i}}^{(k)}\}_{k=1}^K \leftarrow \text{TEACHER\_FORWARD}(\text{forward\_kwargs})$  ▷ Alg. 3
13:       $\mathbf{w}_{\hat{i}} \leftarrow \text{GET\_MOF\_WEIGHTS}(t_{\hat{i}}, \mathcal{B}, \hat{i})$  ▷ router or LUT
14:       $v_{\hat{i}}^{\text{target}} \leftarrow \sum_k \mathbf{w}_{\hat{i}}^{(k)} \cdot v_{\hat{i}}^{(k)}$  ▷ Eq. (28)
15:       $V^{\text{target}}.\text{append}(v_{\hat{i}}^{\text{target}}.\text{detach}())$ 
16:     end for
17:     // === Main pass: student forward + MSE (with grad) ===
18:     for  $\hat{i} \in \mathcal{T}$  do
19:       with accelerator.accumulate( $\theta$ ):
20:          $t_{\hat{i}} \leftarrow \mathcal{B}.\text{timesteps}[:, \hat{i}]$ 
21:          $x_{t_{\hat{i}}} \leftarrow \mathcal{B}.\text{all\_latents}[:, \text{idx\_map}[\hat{i]]$ 
22:         forward_kwargs  $\leftarrow \text{BUILD\_FORWARD\_KWARGS}(\mathcal{B}, t_{\hat{i}}, x_{t_{\hat{i}}})$ 
23:          $v_{\hat{i}}^\theta \leftarrow \text{adapter.forward}(\text{forward\_kwargs}).\text{noise\_pred}$ 
24:          $\mathcal{L} \leftarrow \text{mean}((v_{\hat{i}}^\theta - V_{\hat{i}}^{\text{target}})^2)$  ▷ Eq. (29)
25:         accelerator.backward( $\mathcal{L}$ )
26:         if accelerator.sync_gradients then
27:           clip_grad_norm( $\theta$ , max_grad_norm)
28:           optimizer.step(); optimizer.zero_grad()
29:         end if
30:       end for
31:     end for
32:   end for
33: end for

```

Algorithm 5 MoF Distill Sampling (`distill.py:sample`)

Require: Student adapter (current θ), per-source dataloaders

```
1: adapter.rollout()
2: trajectory_indices  $\leftarrow$  COMPUTETRAJECTORYINDICES(train_ts, T)     $\triangleright$  Stores all needed steps
3: for batch in INTERLEAVEDSOURCEITER do
4:   sample_kwargs  $\leftarrow$  {trajectory_indices, batch, compute_log_prob=False}
5:   samples  $\leftarrow$  adapter.inference(**sample_kwargs)     $\triangleright$  Pure student forward, no MoF
6:   stitch metadata  $\rightarrow$  sample.extra_kwargs
7: end for
8: return samples     $\triangleright$  Each carries all_latents and timesteps
```

5.6 Train Timestep Indices: ODE vs SDE

`distill.py:_train_timestep_indices` 区分两种 dynamics:

- **ODE** (`dynamics_type == "ODE"`): 使用所有 $\{0, 1, \dots, T-1\}$ 个 timestep (确定性轨迹, 每点都可作为 distillation target);
- **SDE / CPS** (Flow-SDE 等): 使用 `scheduler.train_timesteps` 指定的子集 (noisy transition 的 log-prob 在某些 step 上不 well-defined, 需要 scheduler 显式指定可训步)。

5.7 LUT 索引语义陷阱

`distill.py:_get_mof_weights` 的 docstring 强调一个易错点: LUT 的列 0 对应推理 **step 0** = 最高 t = 最噪 **latent**, 列 $T-1$ 对应 **step** $T-1$ = 最低 t = 最干净。因此查表必须用 `loop` 计数器 `timestep_index` ($0..T-1$), 不能用 `t / TIMESTEP_MAX` (这会反向)。

Router 模式无此陷阱: router 接受连续 t 作为输入, 自然处理任意推理步数。

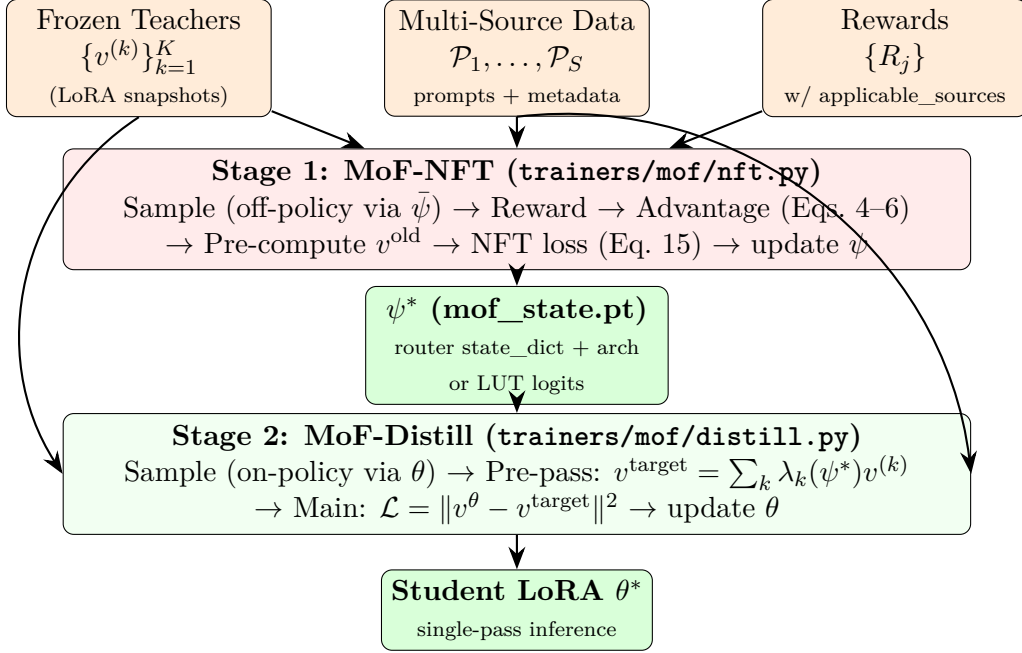
5.8 Checkpoint 兼容性校验

`distill.py:_load_mof_weights` 在加载 `mof_state.pt` 时执行如下校验, 任一失败 fail loud:

1. **Teacher 数量**: `state["K"]` 必须等于当前 `teacher_paths` 长度;
2. **Teacher 顺序**: `state["teacher_names"]` 必须与当前 `_teacher_names position-by-position` 一致——K 轴是 positional 的, 重排会静默把权重应用到错误的 teacher (`common.py:_validate_teacher_o`);
3. **LUT T 一致性**: LUT 模式下 `lut_T` 必须等于 `distill` 配置的 `num_inference_steps`;
4. **Router 架构一致性**: router 模式下, checkpoint 的 `router_arch` (包含 `d_pool`, `d_seq`, `d_hidden`, `d_time`, `tau`) 覆盖 config 漂移, 打印 warning 后采用 checkpoint 值——保证训练时和加载时是同一个网络;
5. **Teacher snapshot 非退化**: Stage 2 启动时 `_verify_teacher_snapshots` 校验每个 teacher 的 LoRA 快照与当前 student weight 的差异 $> 10^{-8}$, 否则报错 (防止 silent LoRA loading failure)。

6 端到端流水线

6.1 流程图



6.2 统一伪代码

Algorithm 6 End-to-End MoF Two-Stage Pipeline

Input: Frozen teachers $\{v^{(k)}\}$, sources $\{\mathcal{P}_s\}$, rewards $\{R_j\}$, base model v^{base}

Output: Student LoRA θ^* that approximates v^{MoF^*}

```

1: // ===== Stage 1: Train router  $\psi$  =====
2: Initialize  $\psi$  (zeros / teacher_biased / random); EMA  $\bar{\psi} \leftarrow \psi$ 
3: for epoch = 1, ...,  $N_{\text{epochs}}^{(1)}$  do
4:   samples  $\leftarrow$  SAMPLE  $\triangleright$  Alg. 2, uses  $\bar{\psi}$ 
5:    $\{R_j(x_b)\}, \{\mathcal{A}_b\} \leftarrow$  COMPUTEREWARDSANDADVANTAGES(samples)  $\triangleright$  Eqs. 4-6
6:   NFTOPTIMIZE(samples,  $\psi, \bar{\psi}$ )  $\triangleright$  Alg. 1
7:    $\bar{\psi} \leftarrow$  EMASSTEP( $\psi$ )
8: end for
9:  $\psi^* \leftarrow \bar{\psi}$  (recommended) or  $\psi$ 
10: Save  $\psi^* + \text{teacher\_names} + \text{router\_arch} \rightarrow \text{mof\_state.pt}$ 

11: // ===== Stage 2: Distill into student  $\theta$  =====
12: Load  $\psi^*$  from mof_state.pt; freeze  $\psi^*$ 
13: Verify teacher snapshots (_verify_teacher_snapshots)
14: Initialize student  $\theta$  (zeros LoRA on top of base)
15: for epoch = 1, ...,  $N_{\text{epochs}}^{(2)}$  do
16:   samples  $\leftarrow$  STUDENTSAMPLE  $\triangleright$  Alg. 5
17:   DISTILLOPTIMIZE(samples,  $\theta, \psi^*$ )  $\triangleright$  Alg. 4
18: end for
19: return  $\theta^*$ 

```

6.3 部署期推理

部署推理 (Stage 2 完成后)

$$x_T \sim \mathcal{N}(0, I) \quad (31)$$

$$\text{for } i = 0, \dots, T-1 : \quad (32)$$

$$v_i = v^{\theta^*}(x_{t_i}, t_i, c) \quad (\text{单次 forward, 不再需要 router / teachers}) \quad (33)$$

$$x_{t_{i+1}} = \text{scheduler.step}(x_{t_i}, v_i, t_i) \quad (34)$$

推理代价: 与单 LoRA 推理相同, 相比 Stage 1 的 K -teacher + router forward, 节省 $\sim K \times \text{FLOPs}$ 。

7 代码 $\Leftrightarrow$ 公式对照表

公式 / 概念	文件	函数 / 行号
$x_{t_i} = (1 - \sigma_i)x_0 + \sigma_i \varepsilon_i$ Eq. (12)	nft.py	optimize, Phase 1 (line ~150)
$\mathcal{M}(v) = \ x_0^{\text{pred}} - x_0\ ^2 / w$ Eq. (14)	nft.py	_nft_weighted_mse
$v^\pm$ Eqs. (10), (11)	nft.py	optimize, Phase 2
$r = \text{clamp}(\mathcal{A} / (2A_{\max}) + 0.5, 0, 1)$ Eq. (7)	nft.py	optimize, Phase 2
NFT total loss Eq. (15)	nft.py	policy_loss computation
3-step advantage Eqs. (4)–(6)	common.py	prepare_feedback
Per-set combined velocity (LUT) Eq. (2)	common.py	_combine_velocities_per_sample
Per-set combined velocity (router)	common.py	_compute_combined_velocity
Sampling with $\sum_k \lambda_k v^{(k)}$	common.py	_mof_inference_context
EMA off-policy split	common.py	sampling_context
Safe teacher forward Alg. 3	common.py	_compute_teacher_velocities
Router-induced teacher Eq. (21)	distill.py	_combine_weighted
Distill loss Eq. (29)	distill.py	optimize, main pass
2-pass design §5.4	distill.py	optimize
Checkpoint validation §5.8	distill.py	_load_mof_weights, _verify_teacher_snapshots
LUT index semantics	distill.py	_get_mof_weights (doc-string)
adaLN router architecture	common.py	MoFAdaLNRouter
LUT (K, T, S) module	common.py	MoFMixingModule
Bypass DDP for weight swap	utils.py	bypass_ddp_for_weight_swap

8 讨论

8.1 Stage 1 与 Stage 2 的训练成本对比

Stage 1 (NFT). 每个 epoch:

- **Sample:** K teacher forwards $\times T$ timesteps $\times$ num_batches_per_epoch

- **Optimize:** K teacher forwards $\times T$ timesteps $\times N_{\text{inner}}$ (Phase 1, no_grad) + T student-style forwards with grad (Phase 2)

Reward evaluation 是额外开销 (每个 sample 跑 $|\mathcal{R}|$ 个 reward model)。可学习参数 $\sim 10^2$ – 10^6 , optimizer state 极小。

Stage 2 (Distill). 每个 epoch:

- **Sample:** T student forwards (生成 trajectory)
- **Optimize:** K teacher forwards (pre-pass) + 1 student forward (main pass) per timestep $\times N_{\text{inner}}$

没有 reward 评估开销, loss 计算便宜 (一个 MSE)。可学习参数 $\sim 10^7$ (LoRA), optimizer state 显著 (AdamW 需要 $2\times$ 参数量的 momentum)。

8.2 常见误区

1. 把 **Stage 2** 的 **trajectory** 用在 **Stage 1** 上: Stage 1 的 **noised_latents** 来自重新采样的高斯噪声对 x_0 加噪 (Eq. (12)), 不是走过的 trajectory 点。这是 NFT 与 GRPO 的本质差异——NFT 是 flow-matching MSE 风格, GRPO 才是 trajectory log-prob ratio 风格。
2. 把 **Stage 1** 的 **sampling EMA** 用在 **Stage 2** 上: Stage 2 的 ψ^* 是完全 **frozen** 的, 没有 EMA / on-policy / off-policy 概念。Student θ 自己决定 trajectory 分布 (on-policy distillation)。
3. 在 **Stage 2** 重做 **advantage**: Stage 2 没有 advantage / reward 概念, 它就是监督式 distillation。
4. **LUT** 模式下用归一化 **timestep** 索引: 必须用 loop 计数器 **timestep_index** ($0..T-1$) 查 LUT 列; 用 $t/1000$ 会反向 (最噪 step 取到最干净 step 的列)。

8.3 何时需要其他变体

- **Stage 1** 用 **GRPO** 而非 **NFT** (`trainers/mof/grpo.py`): 当 dynamics 是 stochastic (CPS / Flow-SDE) 且想用 importance ratio 做 trust region 控制时。Stage 2 不变。
- **Stage 2** 加 **KL penalty**: 在 (30) 上叠加 $\|v^\theta - v^{\text{ref}}\|^2$ 防止 student 偏离 base model 太远, 类似 RLHF 中的 KL 锚定。
- 多阶段迭代: Stage 2 完成后把 θ^* 作为 base, 重新跑 Stage 1 训练一组新的 ψ (更精细的混合), 再 Stage 2 蒸馏——渐进式 reward 改进。

8.4 失败模式与排查

现象	可能原因	排查
所有 teacher 输出相同	autocast cache 或 DDP buffer 未 bypass	检查 weight swap 循环外是否 disable autocast cache + bypass DDP
Stage 2 grad 极小	不会发生 (model weight 空间无 mean-subtraction 瓶颈)	若发生, 检查 v^{target} 是否真的 detach; teacher snapshots 是否退化
Stage 1 grad $\sim 10^{-10}$	LoRA-shared-base 下 normalized-mixing 固有瓶颈	见 <code>mof_per_set_analysis.tex</code> 第 14 节, 考虑切换 Solution A (Direct Reward)
LUT distill 错位	用 <code>t/1000</code> 而非 loop counter 索引 LUT	检查 <code>_get_mof_weights</code> 的 <code>timestep_index</code> 来源
Teacher snapshot 与 student 等同	LoRA 加载失败 (路径错 / 权重格式错)	<code>_verify_teacher_snapshots</code> 启动时报告
Stage 2 reward 不如 Stage 1	Distillation gap (student 容量不足 / 训练不充分)	增加 epoch; 检查 student LoRA rank

9 结论

核心结论

1. 二阶段训练把 **reward** 优化与推理压缩解耦: Stage 1 用极少的 mixing 参数找出 reward-optimal 的 teacher 加权策略; Stage 2 用一个独立的 student LoRA 把这个策略蒸馏成单 forward 推理。
2. **Stage 1 (NFT)** 数学本质: reward-weighted self-normalized MSE on x_0 reconstruction, 配合 EMA 实现 off-policy。 β 在梯度中精确消去, 仅通过 self-norm weight 作二阶 magnitude 调制。
3. **Stage 2 (Distill)** 数学本质: on-policy student trajectory 上的 vanilla velocity MSE, target 由 frozen ψ^* 加权 K 个 teacher 得到 (Eq. (21))。
4. 两阶段共享的硬性约束: teacher forward 必须搭配 `set_autocast_cache_enabled(False)` + `bypass_ddp_for_weight_swap`, 否则所有 teacher 静默退化为同一输出。Stage 2 的 2-pass 设计就是为了把 teacher forward 隔离在 `accelerator.accumulate` 之外, 规避这一约束与 DDP gradient 路径的冲突。
5. **Bridge** 是单向的: ψ^* 在 Stage 2 中完全 frozen; Stage 2 不引入 reward / advantage / EMA / on-off-policy 区分——一旦 ψ^* 给定, Stage 2 退化为标准的 supervised distillation。